Exploratory Data Analysis using Python

Customer Shopping Behavior Analysis | Riya Cherian | Tools: Python, Pandas, Jupyter Notebook, SQLAlchemy

1. Purpose of this stage

Before answering any business question, I used Python to understand the data, find problems in it, and prepare a clean version for SQL and Power BI. This document lists every step I performed in the Jupyter notebook, in order, with the code, the output, and what I learned.

Dataset: customer_shopping_behavior.csv (3,900 rows, 18 columns). Each row is one customer.

2. Summary of the steps

Step	What I did	Result
1	Loaded the CSV into a Pandas DataFrame	3,900 rows, 18 columns
2	Previewed the first rows with `head()`	Understood the columns and values
3	Checked data types and null counts with `info()`	Found that Review Rating has missing values
4	Summarised every column with `describe(include='all')`	Got ranges, averages and most common values
5	Counted missing values with `isnull().sum()`	37 missing ratings
6	Filled missing ratings with the category median	0 missing values
7	Standardised column names	snake_case names, ready for SQL
8	Created `age_group`	4 equal-sized age segments
9	Created `purchase_frequency_days`	Text frequency became numbers

Step	What I did	Result
10	Checked whether two columns were identical	`discount_applied` equals `promo_code_used` in every row
11	Dropped the redundant column	19 final columns
12	Loaded the clean data into MySQL	Table ready for SQL analysis

3. Step-by-step analysis

Step 1: Load the data

```
import pandas as pd
df = pd.read_csv('customer_shopping_behavior.csv')
```

The CSV file was read into a DataFrame called `df`.

Step 2: Preview the data

```
df.head()
```

The first rows showed one customer per row, with columns such as Age, Gender, Item Purchased, Category, Purchase Amount (USD), Location, Size, Color, Season, Review Rating, Subscription Status, Shipping Type, Discount Applied, Promo Code Used, Previous Purchases, Payment Method and Frequency of Purchases.

Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Review Rating	Subscription Status	Shipping Type
1	55	Male	Blouse	Clothing	53	Kentucky	3.1	Yes	Express
2	19	Male	Sweater	Clothing	64	Maine	3.1	Yes	Express
3	50	Male	Jeans	Clothing	73	Massachusetts	3.1	Yes	Free Shipping
4	21	Male	Sandals	Footwear	90	Rhode Island	3.5	Yes	Next Day Air
5	45	Male	Blouse	Clothing	49	Oregon	2.7	Yes	Free Shipping

(Some columns are hidden here to fit the page.)

Step 3: Check structure and data types

```
df.info()
```

What I learned:

- The data has 3,900 rows and 18 columns, using about 548.6 KB of memory.
- 4 columns are integers (Customer ID, Age, Purchase Amount, Previous Purchases), 1 is a decimal (Review Rating), and 13 are text.
- Every column has 3,900 non-null values **except Review Rating, which has only 3,863**. This showed me that some ratings were missing.

Step 4: Summary statistics for every column

```
df.describe(include='all')
```

Numeric columns

Column	Mean	Std. dev.	Min	25%	Median	75%	Max
Age	44.07	15.21	18	31	44	57	70
Purchase Amount (USD)	59.76	23.69	20	39	60	81	100
Review Rating	3.75	0.72	2.5	3.1	3.8	4.4	5.0
Previous Purchases	25.35	14.45	1	13	25	38	50

In every column the mean and the median are very close, so the values are fairly balanced and are not pulled by extreme outliers. Customer IDs run from 1 to 3,900, so each ID appears once.

Text columns

Column	Unique values	Most common	Count
Gender	2	Male	2,652 (68%)
Item Purchased	25	Blouse	171
Category	4	Clothing	1,737 (44.5%)
Location	50	Montana	96
Size	4	M	1,755

Column	Unique values	Most common	Count
Color	25	Olive	177
Season	4	Spring	999
Subscription Status	2	No	2,847 (73%)
Shipping Type	6	Free Shipping	675
Discount Applied	2	No	2,223 (57%)
Promo Code Used	2	No	2,223 (57%)
Payment Method	6	PayPal	677
Frequency of Purchases	7	Every 3 Months	584

Early observations

- The customer base is male-heavy: 2,652 men and 1,248 women.
- Clothing is the largest category.
- Most customers (73%) do not have a subscription.
- 43% of purchases (1,677) used a discount.
- Discount Applied and Promo Code Used have the same count and the same most common value, so they might be duplicates. I tested this in Step 10.

Step 5: Check for missing values

```
df.isnull().sum()
```

Only one column had missing data:

Column	Missing values
Review Rating	**37** (about 1% of rows)
All other 17 columns	0

Step 6: Handle the missing ratings

```
df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(
    lambda x: x.fillna(x.median())
)
df.isnull().sum()
```

Each missing rating was filled with the **median rating of its own category**.

Why this method?

- The **median** is not distorted by unusually high or low ratings.
- **Grouping by category** means a missing Footwear rating is filled using Footwear ratings, not the overall average, so each category keeps its own rating level.
- Dropping the 37 rows would have lost data for no good reason.

Result: `isnull().sum()` now showed 0 missing values in every column.

Step 7: Standardise the column names

```
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})
```

Column names with spaces, capital letters and brackets (such as `Purchase Amount (USD)`) are awkward to use in SQL. After this step the names look like this:

`customer_id, age, gender, item_purchased, category, purchase_amount, location, size, color, season, review_rating, subscription_status, shipping_type, discount_applied, promo_code_used, previous_purchases, payment_method, frequency_of_purchases`

Step 8: Feature engineering - age groups

```
labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
df['age_group'] = pd.qcut(df['age'], q=4, labels=labels)
df[['age', 'age_group']].head(10)
```

`pd.qcut` splits customers into 4 groups with about the same number of people in each, using the age quartiles (31, 44 and 57 from Step 4).

Age group	Age range
Young Adult	18 to 31
Adult	32 to 44
Middle-aged	45 to 57
Senior	58 to 70

Check of the output: age 55 became Middle-aged, 19 and 21 became Young Adult, and 63 became Senior, which matches the ranges above.

Why: equal-sized groups make revenue comparisons between age groups fair.

Step 9: Feature engineering - purchase frequency in days

```
frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}
df['purchase_frequency_days'] =
df['frequency_of_purchases'].map(frequency_mapping)
df[['purchase_frequency_days', 'frequency_of_purchases']].head(10)
```

The text column `frequency_of_purchases` was turned into a number of days between purchases, so it can be averaged and compared.

Text value	Days
Weekly	7
Fortnightly, Bi-Weekly	14
Monthly	30
Quarterly, Every 3 Months	90

Text value	Days
Annually	365

Check of the output: Weekly became 7, Annually became 365, and Quarterly became 90.

Step 10: Check for duplicate information

```
(df['discount_applied'] == df['promo_code_used']).all()
```

Output: `True`

The two columns matched in all 3,900 rows, so `promo_code_used` carries no extra information.

Step 11: Remove the redundant column

```
df = df.drop('promo_code_used', axis=1)
df.columns
```

The final DataFrame has **19 columns**: the 17 original columns that remained, plus `age_group` and `purchase_frequency_days`.

Step 12: Load the clean data into MySQL

```
from getpass import getpass
from sqlalchemy import create_engine

password = getpass('Enter MySQL password: ')   # typed at run time, never
saved in the notebook
engine = create_engine(f"mysql+pymysql://root:
{password}@localhost:3306/customer_behavior")

df.to_sql('customer_shopping_behavior', engine, if_exists='replace',
index=False)
```

The cleaned DataFrame was written to the table `customer_shopping_behavior` in the `customer_behavior` database. I then ran `DESCRIBE customer_shopping_behavior;` in MySQL and confirmed it contained all 19 columns with the correct data types.

4. Before and after

	Before cleaning	After cleaning
Rows	3,900	3,900
Columns	18	19 (1 removed, 2 added)
Missing values	37 (Review Rating)	0
Column names	Spaces, capitals, brackets	snake_case
Age information	Raw ages only	Raw ages plus `age_group`
Purchase frequency	Text only	Text plus `purchase_frequency_days`
Duplicate column	`promo_code_used` present	Removed

5. Key findings from the EDA

1. The dataset is clean apart from 37 missing ratings (about 1%), which were filled with the category median.
2. Customers are 68% male and 32% female.
3. The average purchase is $59.76, and purchases range from $20 to $100.
4. Clothing is the biggest category (44.5% of purchases).
5. 73% of customers have no subscription, and 43% of purchases used a discount.
6. The average rating is about 3.75 out of 5.
7. `promo_code_used` and `discount_applied` are identical, so one was removed.

These findings guided the SQL questions in the next stage, such as how revenue differs by gender and age group, and whether subscribers spend more.

6. Skills shown in this stage

- Data loading and inspection with Pandas (`read_csv` , `head` , `info` , `describe`)
- Missing-value detection and group-based imputation (`isnull` , `groupby` , `transform`)
- Data cleaning and standardisation of column names
- Feature engineering (`pd.qcut` , `map`)
- Data validation (checking that two columns are identical)

- Loading data into a database with SQLAlchemy